Homework 1

Kinematics and Camera Calibration

Sachin Bhadang
PID: A69040722
Email: sbhadang@ucsd.edu

CSE276C, Fall 2026 October 15, 2026

1 Forward and inverse kinematics

1(a): Reference frames and forward kinematics

All lengths in this section are in centimeters. The world origin is the bottom of the fixed base, with $+x$ right, $+y$ up, and $+z$ out of the page. The first joint is at $(0, 5)$. The three moving links each have length $\ell = 10$. Positive joint angles are counterclockwise about $+z$, measured relative to the preceding link; all three angles are zero when the arm points along $+y$. Consequently, negative angles bend a straight arm toward $+x$.

Put frame J_i at joint q_i, with its y_i axis along that joint's outgoing link and its x_i axis completing a right-handed frame. The absolute frame angles are

$$\theta_0 = q_0, \qquad \theta_1 = q_0 + q_1, \qquad \theta_2 = q_0 + q_1 + q_2.$$

In a planar homogeneous representation, define

$$\mathcal{R}(q) = \begin{bmatrix} \cos q & -\sin q & 0 \\ \sin q & \cos q & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad \mathcal{D}(d) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & d \\ 0 & 0 & 1 \end{bmatrix}.$$

The transform from the end-effector frame to the world frame is

$${}^{W}T_{E} = \mathcal{D}(5)\mathcal{R}(q_0)\mathcal{D}(10)\mathcal{R}(q_1)\mathcal{D}(10)\mathcal{R}(q_2)\mathcal{D}(10).$$

A link's displacement is its local vector $(0, 10)^{\mathsf{T}}$ rotated by its absolute angle: $\mathcal{R}_{2\times 2}(\theta)(0, 10)^{\mathsf{T}} = 10(-\sin\theta, \cos\theta)^{\mathsf{T}}$. Adding these three displacements to the fixed base offset gives

$$\boxed{\begin{aligned} x(q) &= -10\,[\sin q_0 + \sin(q_0 + q_1) + \sin(q_0 + q_1 + q_2)], \\ y(q) &= 5 + 10\,[\cos q_0 + \cos(q_0 + q_1) + \cos(q_0 + q_1 + q_2)]. \end{aligned}} \tag{1}$$

As a check, $q = (0, 0, 0)^{\mathsf{T}}$ gives $(x, y) = (0, 35)$.

Differential motion model

Differentiate the position with respect to the three joint angles. Writing $c_i = \cos\theta_i$ and $s_i = \sin\theta_i$,

$$\boxed{\dot{\boldsymbol{x}} = T(q)\dot{q}, \qquad T(q) = -10 \begin{bmatrix} c_0 + c_1 + c_2 & c_1 + c_2 & c_2 \\ s_0 + s_1 + s_2 & s_1 + s_2 & s_2 \end{bmatrix}.} \tag{2}$$

Angles in this derivative are in radians, so T has units cm/rad and $\dot{\boldsymbol{x}}$ has units cm/s when $\dot{q}$ is in rad/s. At the straight pose, $T(0) = \left[\begin{smallmatrix} -30 & -20 & -10 \\ 0 & 0 & 0 \end{smallmatrix}\right]$, which has rank one: instantaneous motion along the extended arm is unavailable.

1(b): A valid inverse-kinematics solution

The desired displacement from the first joint is $(10, 15) - (0, 5) = (10, 10)$. Choose absolute link angles, measured counterclockwise from $+y$, symmetrically about $-\pi/4$:

$$(\theta_0, \theta_1, \theta_2) = \left(\alpha - \frac{\pi}{4}, -\frac{\pi}{4}, -\alpha - \frac{\pi}{4}\right).$$

For the unit direction $d(\theta) = (-\sin\theta, \cos\theta)^{\mathsf{T}}$, the trigonometric sum identities give

$$d(-\pi/4 + \alpha) + d(-\pi/4) + d(-\pi/4 - \alpha) = (1 + 2\cos\alpha)\frac{(1,1)^{\mathsf{T}}}{\sqrt{2}}.$$

Thus each component of the 10-cm-link sum equals 10 exactly if

$$1 + 2\cos\alpha = \sqrt{2}, \qquad \alpha = \arccos\left(\frac{\sqrt{2} - 1}{2}\right) = 78.0471441093^\circ.$$

Taking differences of consecutive absolute angles yields

$$\boxed{q = \begin{bmatrix} \alpha - \pi/4 \\ -\alpha \\ -\alpha \end{bmatrix} = \begin{bmatrix} 33.0471441093^\circ \\ -78.0471441093^\circ \\ -78.0471441093^\circ \end{bmatrix}.} \tag{3}$$

Every joint lies in $[-90^\circ, 90^\circ]$. Substitution into the forward kinematics gives $(x, y) = (10.000000, 15.000000)$ cm, with numerical error below 10^{-12} cm using the unrounded angles.

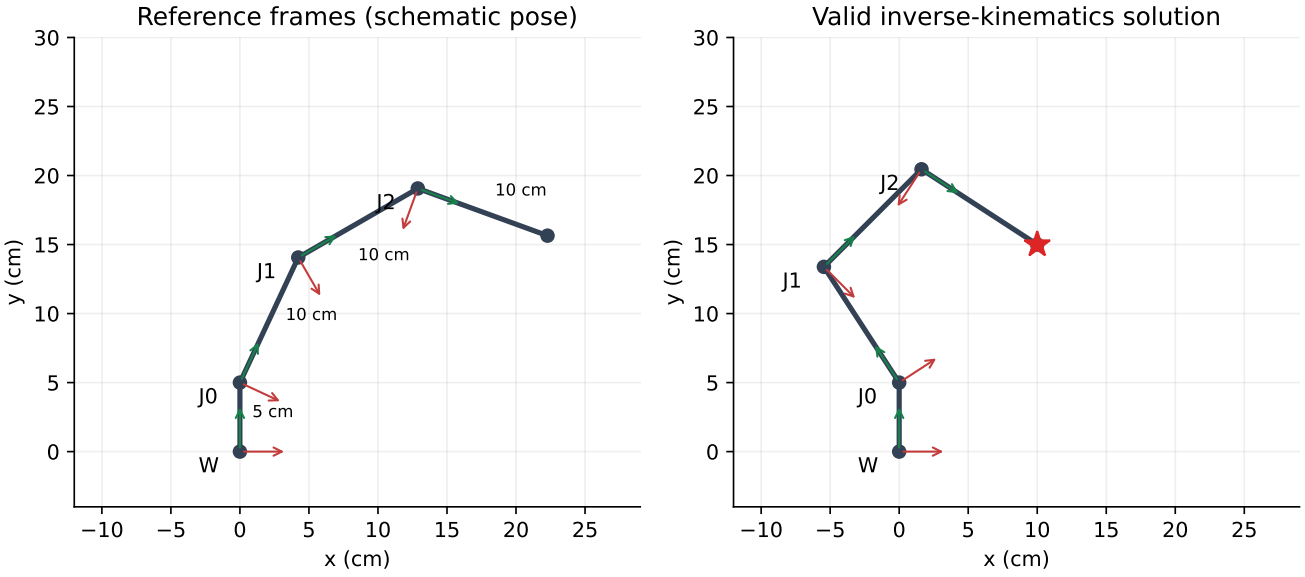


Figure 1: The assignment's arm geometry redrawn with world and joint frames (left), and the computed IK configuration (right). Red arrows are local $+x$ axes; green arrows are local $+y$ axes; every $+z$ axis points out of the page. J_0, J_1, J_2 are the frames at q_0, q_1, q_2. The star marks $(10, 15)$ cm.

At this solution T has rank two and a one-dimensional null space. A unit null vector is

$$n = \begin{bmatrix} 0.4472135955 & -0.6324555320 & 0.6324555320 \end{bmatrix}^{\mathsf{T}}, \qquad Tn = 0.$$

Hence $\dot{q} = T^{+}\dot{\boldsymbol{x}} + \eta n$, with scalar η, realizes the same instantaneous end-effector velocity. This explains the local redundancy of the three-joint arm for a two-coordinate position task; the closed-form construction above supplies the single solution requested.

2 Camera calibration

2.1(a): World coordinates and homography equations

The input contains 432 pairs of pixel coordinates, grouped into nine consecutive images with 48 points each. Within each image, eight points form a left-to-right row and six rows run from top to bottom. With zero-based row $r \in \{0, \dots, 5\}$ and column $c \in \{0, \dots, 7\}$, assign

$$P_{8r+c} = \begin{bmatrix} 25c & 25(5-r) & 0 \end{bmatrix}^{\mathsf{T}} \text{ mm}, \qquad p_{8r+c} = \begin{bmatrix} 25c & 25(5-r) & 1 \end{bmatrix}^{\mathsf{T}}. \tag{4}$$

The origin is the bottom-left *detected corner*, not the outer paper boundary. Thus the first file point has board coordinates $(0, 125)$, point 7 has $(175, 125)$, point 40 has $(0, 0)$, and point 47 has $(175, 0)$ mm. World X points right, Y up the board, and $Z = X \times Y$. Pixel u increases rightward and v downward. The detected-corner grid spans 175×125 mm: eight corners define seven horizontal intervals and six corners define five vertical intervals.

For $p_i = (X_i, Y_i, 1)^{\mathsf{T}}$ and $\tilde{u}_i = (u_i, v_i, 1)^{\mathsf{T}}$, the actual projective relation is $s_i \tilde{u}_i = H p_i$. Use the row-major vectorization

$$h = (h_{11}, h_{12}, h_{13}, h_{21}, h_{22}, h_{23}, h_{31}, h_{32}, h_{33})^{\mathsf{T}}.$$

Eliminating $s_i = h_{31}X_i + h_{32}Y_i + h_{33}$ gives

$$\begin{aligned} h_{11}X_i + h_{12}Y_i + h_{13} - u_i(h_{31}X_i + h_{32}Y_i + h_{33}) = 0, \\ h_{21}X_i + h_{22}Y_i + h_{23} - v_i(h_{31}X_i + h_{32}Y_i + h_{33}) = 0. \end{aligned}$$

Therefore

$$\boxed{M_i = \begin{bmatrix} X_i & Y_i & 1 & 0 & 0 & 0 & -u_iX_i & -u_iY_i & -u_i \\ 0 & 0 & 0 & X_i & Y_i & 1 & -v_iX_i & -v_iY_i & -v_i \end{bmatrix}, \qquad M_i h = 0.} \tag{5}$$

1. Each correspondence supplies **two independent scalar equations**.
2. H has nine entries but is defined only up to nonzero scale, giving **eight degrees of freedom**. Thus eight independent equations, or **four correspondences in general position**, suffice. Collinear or otherwise degenerate point arrangements do not suffice.
3. Each image supplies $2(48) = 96$ equations for those eight degrees of freedom. Its system is **overdetermined**, with $M \in \mathbb{R}^{96 \times 9}$.

2.1(b): Homogeneous least squares

For each view, stack its 48 blocks. The nonzero homogeneous least-squares solution minimizes $\|Mh\|_2$ subject to $\|h\|_2 = 1$ and is the last right singular vector of M. For numerical conditioning, both point sets are first centered and scaled so their mean distance to the origin is $\sqrt{2}$. If $p' = N_p p$ and $u' = N_u u$, solve for H' in these coordinates and recover

$$H = N_u^{-1} H' N_p, \qquad H \leftarrow H / H_{33}.$$

This is a normalized linear DLT estimate, not a nonlinear minimization of pixel error. All calculations use double precision. The nine matrices below are rounded only for display; subsequent calculations use their full precision.

Homography matrices, in image-file order

These matrices map board coordinates in millimeters to homogeneous image coordinates, with $H_{33} = 1$. Reprojection divides by the third coordinate.

$$H_1 = \begin{bmatrix} 1.529753 & 0.495223 & 114.097565 \\ -0.098593 & -0.411171 & 273.823693 \\ 0.000188 & 0.001976 & 1.000000 \end{bmatrix}$$

$$H_2 = \begin{bmatrix} 1.628322 & 0.253506 & 196.146199 \\ -0.015484 & -0.433359 & 275.202229 \\ 0.000568 & 0.001905 & 1.000000 \end{bmatrix}$$

$$H_3 = \begin{bmatrix} 1.768884 & -0.057453 & 148.550326 \\ 0.041759 & -0.652196 & 323.144120 \\ 0.001084 & 0.001649 & 1.000000 \end{bmatrix}$$

$$H_4 = \begin{bmatrix} 1.974559 & -0.260418 & 269.206560 \\ 0.114267 & -0.810967 & 310.181257 \\ 0.001481 & 0.001672 & 1.000000 \end{bmatrix}$$

$$H_5 = \begin{bmatrix} 1.789825 & -0.232785 & 225.412112 \\ 0.110863 & -0.728106 & 381.443616 \\ 0.001374 & 0.001515 & 1.000000 \end{bmatrix}$$

$$H_6 = \begin{bmatrix} 1.782385 & -0.373168 & 248.937937 \\ 0.168182 & -0.771653 & 337.649892 \\ 0.001601 & 0.001374 & 1.000000 \end{bmatrix}$$

$$H_7 = \begin{bmatrix} 1.829844 & -0.379205 & 175.973039 \\ 0.185915 & -0.781881 & 392.693150 \\ 0.001697 & 0.001400 & 1.000000 \end{bmatrix}$$

$$H_8 = \begin{bmatrix} 1.725898 & -0.385103 & 325.095087 \\ -0.032442 & -0.829978 & 325.513611 \\ 0.001346 & 0.001273 & 1.000000 \end{bmatrix}$$

$$H_9 = \begin{bmatrix} 1.720368 & -0.279565 & 321.685119 \\ 0.142670 & -0.576736 & 322.016662 \\ 0.001403 & 0.001563 & 1.000000 \end{bmatrix}$$

Reprojection errors

For every view j and corner i, compute the Euclidean pixel error

$$e_{ji} = \left\| \begin{bmatrix} u_{ji} \\ v_{ji} \end{bmatrix} - \pi(H_j p_i) \right\|_2, \qquad \pi(a, b, c) = (a/c, b/c)^{\mathsf{T}}.$$

The requested total is $\boxed{\sum_{j,i} e_{ji} = 162.931123 \text{ pixels}}$. Across all 432 corners, the mean is 0.377155 pixels and the pointwise RMS is $\sqrt{\sum e_{ji}^2/432} = 0.434518$ pixels. For clarity, the sum of *squared* errors is a different quantity: 81.563984 pixels^2.

View	Sum (px)	Mean (px)	RMS (px)	Maximum (px)
1	20.409515	0.425198	0.487243	0.934292
2	18.998046	0.395793	0.456896	1.001749
3	17.915610	0.373242	0.424739	0.912635
4	20.019560	0.417074	0.474302	0.955674
5	16.041796	0.334204	0.392116	0.869267
6	17.357986	0.361625	0.417608	0.884603
7	16.904627	0.352180	0.410637	0.832270
8	16.594989	0.345729	0.396308	0.795087
9	18.688993	0.389354	0.440330	0.894420

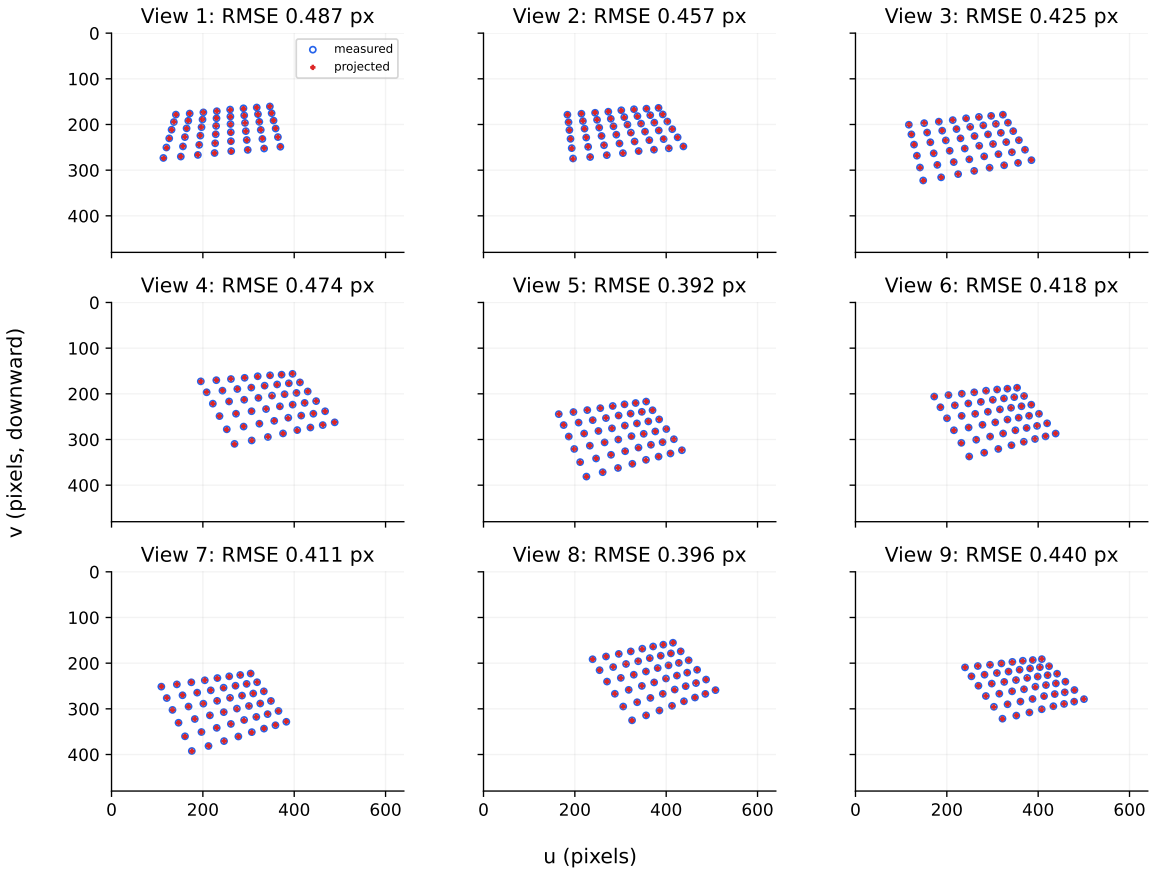


Figure 2: Measured corners and normalized-DLT reprojections for all nine views. The subpixel discrepancies are small at the scale of the full 640×480 images.

2.2(a–b): Constraints on the intrinsic matrix

Let $H = [h_1\ h_2\ h_3]$ and $B = A^{-\mathsf{T}}A^{-1}$. Use the *assignment's* vector order

$$b = (b_{11}, b_{12}, b_{13}, b_{22}, b_{23}, b_{33})^{\mathsf{T}}, \qquad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{12} & b_{22} & b_{23} \\ b_{13} & b_{23} & b_{33} \end{bmatrix}.$$

For column vectors $h_i = (h_{1i}, h_{2i}, h_{3i})^{\mathsf{T}}$, direct expansion gives $h_i^{\mathsf{T}} B h_j = v_{ij}^{\mathsf{T}} b$, where

$$v_{ij} = \begin{bmatrix} h_{1i}h_{1j} \\ h_{1i}h_{2j} + h_{2i}h_{1j} \\ h_{1i}h_{3j} + h_{3i}h_{1j} \\ h_{2i}h_{2j} \\ h_{2i}h_{3j} + h_{3i}h_{2j} \\ h_{3i}h_{3j} \end{bmatrix}.$$

The orthogonality and equal-length constraints therefore become

$$\boxed{L_j b = 0, \qquad L_j = \begin{bmatrix} v_{12}^{\mathsf{T}} \\ (v_{11} - v_{22})^{\mathsf{T}} \end{bmatrix}, \qquad L = \begin{bmatrix} L_1 \\ \vdots \\ L_9 \end{bmatrix}.} \tag{6}$$

1. Each homography supplies **two scalar constraints**.
2. A general symmetric B has six entries, or five degrees of freedom after scale is removed. Thus **three suitably different views** are needed for the unconstrained six-vector solution. If zero skew is imposed, $b_{12} = 0$ removes another degree of freedom, and **two nondegenerate views** can suffice. Merely translating a board at fixed orientation does not supply the required diversity.
3. With nine views, L is 18×6, so this is **overdetermined** for either model.

For the specified six-vector solve, the last right singular vector of L gives b_{free}. To also enforce the assignment's opening assumption of no skew, remove the column multiplying b_{12}, solve the resulting 18×5 system, and insert $b_{12} = 0$ exactly. This gives b_0. Both are normalized to $b_{33} = 1$ below:

$$b_{\text{free}} = \begin{bmatrix} 1.887510 \times 10^{-6} \\ -4.296401 \times 10^{-8} \\ -6.328265 \times 10^{-4} \\ 1.863009 \times 10^{-6} \\ -4.046908 \times 10^{-4} \\ 1.000000 \times 10^{0} \end{bmatrix}, \qquad b_0 = \begin{bmatrix} 1.916084 \times 10^{-6} \\ 0.000000 \\ -6.314704 \times 10^{-4} \\ 1.746239 \times 10^{-6} \\ -4.677439 \times 10^{-4} \\ 1.000000 \times 10^{0} \end{bmatrix}.$$

Both reconstructed B matrices are positive definite. The six singular values of L are approximately

$$(9.27759, 3.83942, 0.473113, 0.00611026, 0.000445818, 2.78357 \times 10^{-7}).$$

Noise makes the smallest value nonzero; its right singular vector supplies the least-squares solution.

2.2(c–d): Recovering and interpreting A

With $d = b_{11}b_{22} - b_{12}^2$, the intrinsic recovery formulas are

$$c_y = \frac{b_{12}b_{13} - b_{11}b_{23}}{d}, \qquad \lambda_B = b_{33} - \frac{b_{13}^2 + c_y(b_{12}b_{13} - b_{11}b_{23})}{b_{11}},$$
$$f_x = \sqrt{\lambda_B / b_{11}}, \qquad f_y = \sqrt{\lambda_B b_{11}/d},$$
$$\gamma = -\frac{b_{12}f_x^2 f_y}{\lambda_B}, \qquad c_x = \frac{\gamma c_y}{f_y} - \frac{b_{13}f_x^2}{\lambda_B}.$$

These follow the closed-form method and Appendix B of Zhang's report [2]; its vector ordering differs from the one required here. The general six-vector solution yields

$$A_{\text{free}} = \begin{bmatrix} 606.149831 & 13.891421 & 340.393784 \\ 0.000000 & 610.282723 & 225.074278 \\ 0.000000 & 0.000000 & 1.000000 \end{bmatrix}.$$

Its estimated skew is 13.891421 pixels. To satisfy the stipulated zero-skew model, the remaining calculations instead use the constrained fit

$$\boxed{A = A_0 = \begin{bmatrix} 589.828760 & 0.000000 & 329.562979 \\ 0.000000 & 617.847616 & 267.857868 \\ 0.000000 & 0.000000 & 1.000000 \end{bmatrix}.} \tag{7}$$

This is obtained by re-solving with $b_{12} = 0$, not by changing just one entry of A_{free} after fitting.

Entry	Estimate	Meaning
$A_{11} = f_x$	589.828760 px	Horizontal focal scale
$A_{22} = f_y$	617.847616 px	Vertical focal scale
$A_{13} = c_x$	329.562979 px	Principal point, horizontal coordinate
$A_{23} = c_y$	267.857868 px	Principal point, vertical coordinate
$A_{12} = \gamma$	0	Enforced zero skew
A_{21}, A_{31}, A_{32}	0	Canonical upper-triangular camera convention
A_{33}	1	Homogeneous normalization

Relative to the nominal image center $(320, 240)$, the principal point is shifted by $(9.563, 27.858)$ pixels. Using zero-based pixel-center coordinates would place the geometric center at $(319.5, 239.5)$ instead. The focal-scale ratio is $f_x/f_y \approx 0.954651$.

A centered camera with square pixels would suggest $c_x \approx 320$, $c_y \approx 240$, and $f_x \approx f_y$. However, the ideal pinhole model itself does *not* require an exactly centered principal point or equal focal scales: unequal pixel spacing or image resizing can produce unequal f_x, f_y. Possible estimation discrepancies include corner localization error, finite and similar viewing directions, checkerboard flatness or printing error, unmodeled lens distortion, and sensor/lens misalignment. The visible change between the constrained and unconstrained estimates is evidence of sensitivity to the modeling constraint, not proof of physical sensor skew. Ground-truth intrinsics are not supplied.

Only the linear calibration steps requested are used; no lens-distortion parameters or nonlinear refinement are fitted.

2.3(a): Extrinsic recovery

For each H, compute $Q = A^{-1}H = [a_1\ a_2\ a_3]$. Ideally the first two columns have equal norms. With noisy estimates, use their symmetric scale

$$\rho = \frac{2}{\|a_1\| + \|a_2\|}, \qquad \tilde{r}_1 = \rho a_1, \quad \tilde{r}_2 = \rho a_2, \quad \tilde{r}_3 = \tilde{r}_1 \times \tilde{r}_2, \quad t = \rho a_3.$$

Choose the sign of ρ so the board origin has positive camera depth. To enforce a proper rotation, take $\tilde{R} = U\Sigma V^{\mathsf{T}}$ and set

$$R = U\,\mathrm{diag}(1, 1, \det(UV^{\mathsf{T}}))V^{\mathsf{T}}.$$

This applies the Section 3.1 pose construction and Appendix C rotation projection [2], with a determinant correction to ensure $\det R = +1$. All matrices below map *world to camera*: $X_c = RX_w + t$. Translations are in millimeters; R is dimensionless.

$$[R_1 \mid t_1] = \begin{bmatrix} 0.99328 & -0.11174 & -0.03027 & -145.66812 \\ -0.09227 & -0.60628 & -0.78988 & 3.85037 \\ 0.06991 & 0.78736 & -0.61252 & 398.76120 \end{bmatrix}$$

$$[R_2 \mid t_2] = \begin{bmatrix} 0.96735 & -0.24739 & 0.05504 & -89.64586 \\ -0.11011 & -0.60584 & -0.78793 & 4.71106 \\ 0.22827 & 0.75615 & -0.61330 & 396.31976 \end{bmatrix}$$

$$[R_3 \mid t_3] = \begin{bmatrix} 0.90219 & -0.39205 & 0.17989 & -116.16425 \\ -0.14842 & -0.67372 & -0.72393 & 33.87079 \\ 0.40501 & 0.62642 & -0.66600 & 378.52060 \end{bmatrix}$$

$$[R_4 \mid t_4] = \begin{bmatrix} 0.85029 & -0.45981 & 0.25608 & -34.50038 \\ -0.15682 & -0.68580 & -0.71069 & 23.09540 \\ 0.50240 & 0.56414 & -0.65524 & 337.15247 \end{bmatrix}$$

$$[R_5 \mid t_5] = \begin{bmatrix} 0.84583 & -0.46433 & 0.26262 & -65.80242 \\ -0.15361 & -0.68346 & -0.71364 & 68.50897 \\ 0.51086 & 0.56328 & -0.64942 & 372.65328 \end{bmatrix}$$

$$[R_6 \mid t_6] = \begin{bmatrix} 0.78912 & -0.51999 & 0.32696 & -50.73176 \\ -0.15655 & -0.68498 & -0.71155 & 41.92377 \\ 0.59395 & 0.51031 & -0.62193 & 371.13839 \end{bmatrix}$$

$$[R_7 \mid t_7] = \begin{bmatrix} 0.78003 & -0.52666 & 0.33791 & -94.44216 \\ -0.15125 & -0.68269 & -0.71488 & 73.27993 \\ 0.60719 & 0.50651 & -0.61218 & 362.68456 \end{bmatrix}$$

$$[R_8 \mid t_8] = \begin{bmatrix} 0.82339 & -0.51020 & 0.24843 & -2.86115 \\ -0.24378 & -0.71335 & -0.65704 & 35.24724 \\ 0.51244 & 0.48044 & -0.71175 & 377.71473 \end{bmatrix}$$

$$[R_9 \mid t_9] = \begin{bmatrix} 0.82370 & -0.51025 & 0.24732 & -5.13871 \\ -0.14957 & -0.61623 & -0.77323 & 33.72559 \\ 0.54695 & 0.59992 & -0.58390 & 384.74411 \end{bmatrix}$$

2.3(b): Camera trajectory

The world-coordinate camera center is the point that maps to zero in camera coordinates. Thus

$$RC + t = 0 \quad \Longrightarrow \quad \boxed{C = -R^{-1}t = -R^{\mathsf{T}}t}.$$

The resulting centers, relative to the bottom-left detected board corner, are

View	C_X (mm)	C_Y (mm)	C_Z (mm)
1	117.1667	-327.9113	242.8800
2	-3.2290	-318.9985	251.7091
3	-43.4764	-259.8366	297.5130
4	-136.4297	-190.2251	246.1642
5	-124.1907	-193.6374	308.1804
6	-173.8430	-187.0587	277.2412
7	-135.4673	-183.4166	306.3272
8	-182.6076	-157.7843	292.7073
9	-201.1592	-212.6542	252.0020

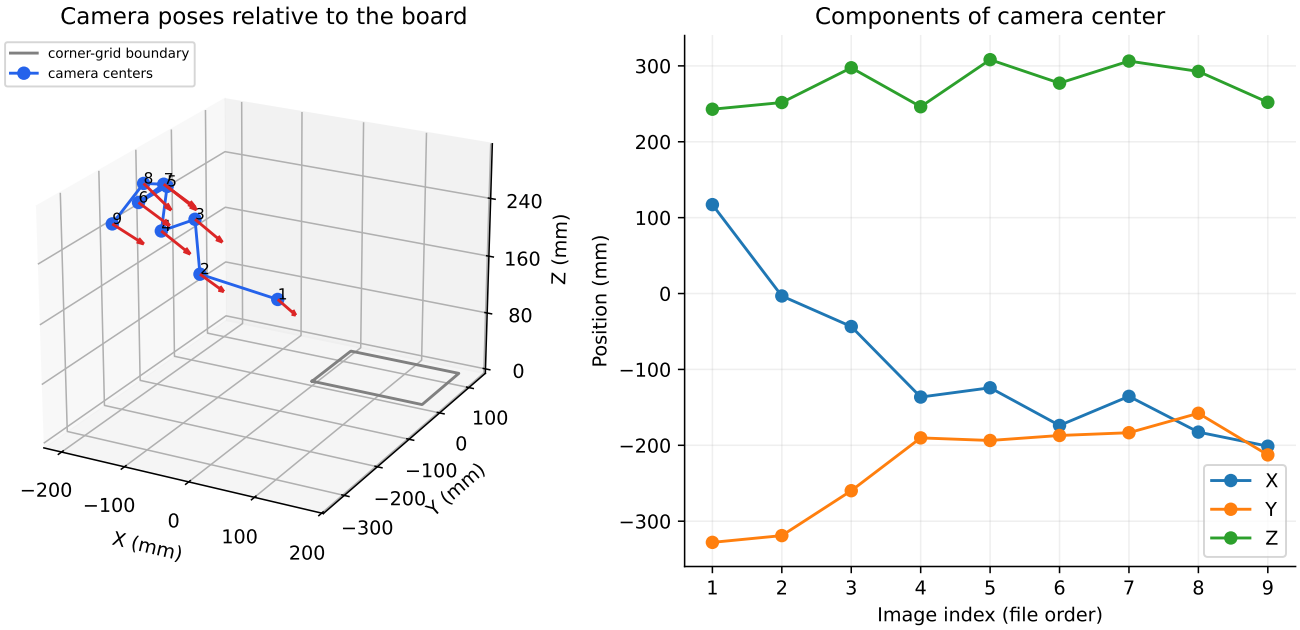


Figure 3: Recovered camera centers and optical directions in the checkerboard frame (left), and coordinate values by view (right). Red arrows point along each camera's forward optical axis, $R^{\mathsf{T}}(0,0,1)^{\mathsf{T}}$.

The camera lies on the positive-Z side of the checkerboard, with recovered board-normal distances between 242.880 and 308.180 mm. Its center moves mainly toward decreasing X, with several reversals visible in the other coordinates. The polyline joins the poses in file order; without timestamps, it does not determine speed or a continuous motion path. These are camera poses *relative to the board*. Interpreting them as the camera's motion in a fixed external world assumes the board stayed fixed.

3 Numerical checks and reproducibility

The code in Appendix A generates every numerical result and figure used in this report. The input is the supplied `imgpoints.txt`. The original `q1_kinematics.py` is not required to run the submitted solution.

The implementation includes the following checks:

- The analytic Jacobian agrees with central finite differences at 30 joint configurations sampled within the permitted range, using step 10^{-6} radians and comparison tolerance 10^{-7}.
- The reported IK configuration respects every joint limit and places the end effector within 10^{-12} cm of the desired point.
- Synthetic noiseless points from four known camera poses recover their homographies, intrinsic parameters, and extrinsics. This is repeated for known skew values of zero and two pixels, exercising both calibration modes. Intrinsic estimates agree with the known camera within 10^{-8} pixels. Both signs of each synthetic homography give the same positive-depth pose.
- The two intrinsic recovery methods—the explicit formulas above and an independent Cholesky factorization of B—agree within 10^{-9}.
- All nine estimated rotations satisfy $R^{\mathsf{T}}R = I$ and $\det R = 1$ to numerical precision. The maximum observed Frobenius norm $\|R^{\mathsf{T}}R - I\|_F$ is 3.28×10^{-15}, and every center satisfies $RC + t = 0$.
- All 432 board points have positive depth in their respective cameras; the minimum depth is 337.152468 mm.

As a further diagnostic, reprojection through the final common A_0 and the orthonormalized $[R_j \mid t_j]$ has RMS error 0.916839 pixels, mean error 0.798639 pixels, and maximum error 2.156184 pixels. These are *calibrated-pose* errors, distinct from the 0.434518-pixel RMS of the nine independently fitted homographies. The shared camera model and rotation constraints make the fits less flexible; the linear estimates have not been refined to minimize pixel error.

To reproduce the numerical work from this directory:

```
python -m pip install -r requirements.txt
python solve_hw1.py
tectonic report.tex
```

An existing Python environment with NumPy and Matplotlib can run the second command directly. A standard LaTeX installation can also compile `report.tex` with `pdflatex` (run twice for cross-references). The execution here used Python 3.13.11, NumPy 2.5.1, Matplotlib 3.11.1, and Tectonic 0.15.0.

`results/summary.json` contains the numerical summary; `results/calibration.npz` contains full-precision matrices, world and image coordinates, and residuals. Numerical tables are generated as LaTeX fragments in `results/`, and the three figures are saved there as vector PDFs. No random data affect the fitted calibration; the fixed random seed is used only for Jacobian checks.

References

[1] CSE276C, *Homework 1: Kinematics and Camera Calibration*, Fall 2026. Supplied assignment and `imgpoints.txt`.

[2] Z. Zhang, *A Flexible New Technique for Camera Calibration*, Microsoft Research Technical Report MSR-TR-98-71, 1998, updated 2008. Sections 2–3.1 and Appendices A–C. Technical report PDF.

A Complete code used for the assignment

The listing is included directly from `solve_hw1.py`; it matches the runnable source. It contains all calculations, validation, plots, and table exports, and requires NumPy and Matplotlib.

```python
"""CSE276C HW1: reproducible kinematics and linear camera calibration.

Run: python solve_hw1.py
Dependencies: numpy, matplotlib. Input: imgpoints.txt beside this script.
Outputs: results/*.pdf, results/calibration.npz, results/summary.json,
and numerical LaTeX fragments used by report.tex.
"""
from pathlib import Path
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parent
OUT = ROOT / "results"


def fk_points(q):
    directions = np.c_[-np.sin(np.cumsum(q)), np.cos(np.cumsum(q))]
    return np.vstack(([0., 0.], [0., 5.],
                      [0., 5.] + 10. * np.cumsum(directions, axis=0)))


def jacobian(q):
    a = np.cumsum(q)
    return -10. * np.array([np.cumsum(np.cos(a)[::-1])[::-1],
                            np.cumsum(np.sin(a)[::-1])[::-1]])


def inverse_kinematics():
    a = np.arccos((np.sqrt(2.) - 1.) / 2.)
    return np.array([a - np.pi / 4., -a, -a])


def world_points():
    # File order: eight points across each row, then the next row down.
    # Origin: bottom-left detected corner; X right, Y up, Z = 0 (mm).
    return np.array([(25. * c, 25. * (5 - r))
                     for r in range(6) for c in range(8)])


def homogeneous(p):
    return np.c_[p, np.ones(len(p))]


def project(H, p):
    z = homogeneous(p) @ H.T
    return z[:, :2] / z[:, 2, None]


def normalize(p):
    center = p.mean(axis=0)
    scale = np.sqrt(2.) / np.linalg.norm(p - center, axis=1).mean()
    N = np.array([[scale, 0., -scale * center[0]],
                  [0., scale, -scale * center[1]], [0., 0., 1.]])
    return project(N, p), N


def homography(p, uv):
    pn, Np = normalize(p)
    un, Nu = normalize(uv)
    ph = homogeneous(pn)
    M = np.zeros((2 * len(p), 9))
    M[0::2, :3] = ph
    M[1::2, 3:6] = ph
    M[0::2, 6:] = -un[:, 0, None] * ph
```

```
    M[1::2, 6:] = -un[:, 1, None] * ph
    _, singular, Vt = np.linalg.svd(M, full_matrices=(len(M) < 9))
    H = np.linalg.solve(Nu, Vt[-1].reshape(3, 3)) @ Np
    return H / H[2, 2], singular


def vrow(H, i, j):
    # Assignment order: b11, b12, b13, b22, b23, b33.
    x, y = H[:, i], H[:, j]
    return np.array([x[0] * y[0], x[0] * y[1] + x[1] * y[0],
                     x[0] * y[2] + x[2] * y[0], x[1] * y[1],
                     x[1] * y[2] + x[2] * y[1], x[2] * y[2]])


def intrinsic(Hs, zero_skew=False):
    L = np.vstack([row for H in Hs for row in
                   (vrow(H, 0, 1), vrow(H, 0, 0) - vrow(H, 1, 1))])
    cols = [0, 2, 3, 4, 5] if zero_skew else list(range(6))
    _, singular, Vt = np.linalg.svd(L[:, cols], full_matrices=False)
    b = np.zeros(6)
    b[cols] = Vt[-1]
    b /= b[-1]  # Report scale: B33 = 1.
    b11, b12, b13, b22, b23, b33 = b
    B = np.array([[b11, b12, b13], [b12, b22, b23],
                  [b13, b23, b33]])
    assert np.linalg.eigvalsh(B).min() > 0., "B must be positive definite"
    d = b11 * b22 - b12**2
    cy = (b12 * b13 - b11 * b23) / d
    lam = b33 - (b13**2 + cy * (b12 * b13 - b11 * b23)) / b11
    fx = np.sqrt(lam / b11)
    fy = np.sqrt(lam * b11 / d)
    skew = -b12 * fx**2 * fy / lam
    cx = skew * cy / fy - b13 * fx**2 / lam
    A = np.array([[fx, skew, cx], [0., fy, cy], [0., 0., 1.]])
    # Independent recovery from B = U.T @ U, where U is upper triangular.
    Ac = np.linalg.inv(np.linalg.cholesky(B).T)
    np.testing.assert_allclose(A, Ac / Ac[2, 2], atol=1e-9, rtol=0)
    return A, b, L, singular


def extrinsic(H, A):
    Q = np.linalg.solve(A, H)
    # Symmetric estimate of the common scale; choose positive board depth.
    scale = 2. / (np.linalg.norm(Q[:, 0]) + np.linalg.norm(Q[:, 1]))
    scale *= np.sign(Q[2, 2])
    r1, r2, t = (scale * Q).T
    R0 = np.column_stack((r1, r2, np.cross(r1, r2)))
    U, _, Vt = np.linalg.svd(R0)
    R = U @ np.diag([1., 1., np.linalg.det(U @ Vt)]) @ Vt
    return R, t, -R.T @ t


def error_stats(errors):
    return dict(total=float(errors.sum()), mean=float(errors.mean()),
                rmse=float(np.sqrt(np.mean(errors**2))),
                sse=float(np.sum(errors**2)), maximum=float(errors.max()))


def validate_math():
    """Check derivatives and recover known noiseless synthetic cameras."""
    rng = np.random.default_rng(276)
    eps = 1e-6
    for q in rng.uniform(-np.pi / 2., np.pi / 2., (30, 3)):
        Jfd = np.column_stack([
            (fk_points(q + eps * e)[-1] - fk_points(q - eps * e)[-1])
            / (2. * eps) for e in np.eye(3)])
        np.testing.assert_allclose(jacobian(q), Jfd, atol=1e-7, rtol=0)
    q = inverse_kinematics()
    assert np.all(np.abs(q) <= np.pi / 2.)
    np.testing.assert_allclose(fk_points(q)[-1], [10., 15.], atol=1e-12,
                               rtol=0)
    p = world_points()
    for skew in (0., 2.):
```

```
        A = np.array([[700., skew, 320.], [0., 720., 240.], [0., 0., 1.]])
        Hs, poses = [], []
        for ax, ay in [(.2, .3), (-.4, .1), (.1, -.5), (.5, .4)]:
            Rx = np.array([[1., 0., 0.], [0., np.cos(ax), -np.sin(ax)],
                           [0., np.sin(ax), np.cos(ax)]])
            Ry = np.array([[np.cos(ay), 0., np.sin(ay)], [0., 1., 0.],
                           [-np.sin(ay), 0., np.cos(ay)]])
            R, t = Rx @ Ry, np.array([20., -40., 600.])
            H = A @ np.column_stack((R[:, :2], t))
            Hfit, _ = homography(p, project(H, p))
            np.testing.assert_allclose(Hfit, H / H[2, 2], atol=1e-10,
                                       rtol=0)
            corners = p[[0, 7, 40, 47]]
            Hfour, _ = homography(corners, project(H, corners))
            np.testing.assert_allclose(Hfour, Hfit, atol=1e-10, rtol=0)
            Hs.append(Hfit)
            poses.append((R, t))
        Afit, _, _, _ = intrinsic(Hs, zero_skew=(skew == 0.))
        np.testing.assert_allclose(Afit, A, atol=1e-8, rtol=0)
        for H, (R, t) in zip(Hs, poses):
            for sign in (-1., 1.):
                Rfit, tfit, _ = extrinsic(sign * H, Afit)
                np.testing.assert_allclose(Rfit, R, atol=1e-10, rtol=0)
                np.testing.assert_allclose(tfit, t, atol=1e-8, rtol=0)


def save_figures(p, uv, Hs, errors, Rs, centers):
    plt.rcParams.update({"font.size": 10, "axes.spines.top": False,
                         "axes.spines.right": False, "savefig.bbox": "tight"})
    fig, axes = plt.subplots(1, 2, figsize=(9, 4.2))
    for ax, q, title in zip(axes, [np.radians([-25., -35., -50.]),
                                   inverse_kinematics()],
                            ["Reference frames (schematic pose)",
                             "Valid inverse-kinematics solution"]):
        pts = fk_points(q)
        ax.plot(*pts.T, "o-", color="#334155", lw=2.5)
        angles = np.r_[0., np.cumsum(q)]
        for k, (origin, angle) in enumerate(zip(pts[:4], angles)):
            for d, color in [(np.array([np.cos(angle), np.sin(angle)]),
                              "#c43b3b"),
                             (np.array([-np.sin(angle), np.cos(angle)]),
                              "#17834b")]:
                ax.annotate("", origin + 3.5 * d, origin,
                            arrowprops=dict(arrowstyle="->", color=color))
            ax.annotate("W" if k == 0 else f"J{k-1}", origin,
                        xytext=(-20, -10), textcoords="offset points")
        for j in range(4) if ax is axes[0] else []:
            midpoint = (pts[j] + pts[j + 1]) / 2.
            ax.annotate("5 cm" if j == 0 else "10 cm", midpoint,
                        xytext=[(6, 0), (8, 0), (0, -18), (6, 8)][j],
                        textcoords="offset points", fontsize=8)
        if ax is axes[1]:
            ax.plot(10., 15., "*", ms=13, color="#dc2626")
        ax.set(xlim=(-12, 29), ylim=(-4, 30), xlabel="x (cm)",
               ylabel="y (cm)", title=title, aspect="equal")
        ax.grid(alpha=.2)
    fig.tight_layout()
    fig.savefig(OUT / "kinematics.pdf")
    plt.close(fig)

    fig, axes = plt.subplots(3, 3, figsize=(10, 7.3), sharex=True, sharey=True)
    for i, ax in enumerate(axes.flat):
        pred = project(Hs[i], p)
        ax.scatter(*uv[i].T, s=13, facecolors="none", edgecolors="#2563eb",
                   label="measured")
        ax.scatter(*pred.T, s=10, marker="+", color="#dc2626", label="projected")
        ax.set(title=f"View {i+1}: RMSE {error_stats(errors[i])['rmse']:.3f} px",
               xlim=(0, 640), ylim=(480, 0), aspect="equal")
        ax.grid(alpha=.15)
    axes[0, 0].legend(fontsize=7, loc="upper right")
    fig.supxlabel("u (pixels)")
    fig.supylabel("v (pixels, downward)")
    fig.tight_layout()
```

```
    fig.savefig(OUT / "reprojections.pdf")
    plt.close(fig)

    fig = plt.figure(figsize=(10, 4.8))
    ax = fig.add_subplot(121, projection="3d")
    border = np.array([[0, 0, 0], [175, 0, 0], [175, 125, 0],
                       [0, 125, 0], [0, 0, 0]])
    ax.plot(*border.T, color="0.5", label="corner-grid boundary")
    ax.plot(*centers.T, "o-", color="#2563eb", label="camera centers")
    for i, (C, R) in enumerate(zip(centers, Rs), 1):
        ax.text(*C, str(i), fontsize=8)
        direction = R.T[:, 2]  # Optical +z axis expressed in board frame.
        ax.quiver(*C, *direction, length=55., color="#dc2626",
                  arrow_length_ratio=.2)
    ax.set(xlabel="X (mm)", ylabel="Y (mm)", zlabel="Z (mm)",
           title="Camera poses relative to the board")
    all_pts = np.vstack((border, centers))
    ax.set_box_aspect(np.ptp(all_pts, axis=0))
    for axis in (ax.xaxis, ax.yaxis, ax.zaxis):
        axis.set_major_locator(plt.MaxNLocator(5))
    ax.view_init(elev=22., azim=-62.)
    ax.legend(fontsize=7, loc="upper left")
    ax = fig.add_subplot(122)
    for k, name in enumerate("XYZ"):
        ax.plot(range(1, 10), centers[:, k], "o-", label=name)
    ax.set(xlabel="Image index (file order)", ylabel="Position (mm)",
           title="Components of camera center", xticks=range(1, 10))
    ax.legend()
    ax.grid(alpha=.2)
    fig.tight_layout()
    fig.savefig(OUT / "trajectory.pdf")
    plt.close(fig)


def tex_matrix(A, digits=6, scientific=False):
    fmt = f".{digits}e" if scientific else f".{digits}f"
    def cell(x):
        if not scientific or x == 0.:
            return format(x if x != 0. else 0., f".{digits}f")
        mantissa, power = format(x, fmt).split("e")
        return mantissa + r"\!\times\!10^{" + str(int(power)) + "}"
    rows = [" & ".join(cell(x) for x in row) for row in np.atleast_2d(A)]
    return r"\begin{bmatrix}" + r" \\ ".join(rows) + r"\end{bmatrix}"


def write_tables(Hs, Rs, ts, centers, errors, A, b, Afree, bfree):
    pieces = [r"\begin{align*}"]
    for i, H in enumerate(Hs, 1):
        pieces.append(f"H_{{{i}}} &= " + tex_matrix(H) + r" \\")
    pieces.append(r"\end{align*}")
    (OUT / "homographies.tex").write_text("\n".join(pieces))
    for start in (0, 5):
        pieces = [r"\begin{align*}"]
        for j in range(start, min(start + 5, 9)):
            E = np.column_stack((Rs[j], ts[j]))
            pieces.append(f"[R_{{{j+1}}}\\mid t_{{{j+1}}}] &= "
                          + tex_matrix(E, 5) + r" \\")
        pieces.append(r"\end{align*}")
        (OUT / f"extrinsics_{start}.tex").write_text("\n".join(pieces))
    rows = []
    for i, (C, e) in enumerate(zip(centers, errors), 1):
        s = error_stats(e)
        rows.append(f"{i} & {s['total']:.6f} & {s['mean']:.6f} & "
                    f"{s['rmse']:.6f} & {s['maximum']:.6f} " + r"\\")
    header = (r"\begin{tabular}{rrrrr}\toprule" + "\n"
              r"View & Sum (px) & Mean (px) & RMS (px) & Maximum (px)"
              r"\\\midrule" + "\n")
    footer = "\n" + r"\bottomrule\end{tabular}" + "\n"
    (OUT / "errors.tex").write_text(header + "\n".join(rows) + footer)
    rows = [str(i) + " & " + " & ".join(f"{x:.4f}" for x in C) + r" \\"
            for i, C in enumerate(centers, 1)]
    header = (r"\begin{tabular}{rrrr}\toprule" + "\n"
              r"View & $C_X$ (mm) & $C_Y$ (mm) & $C_Z$ (mm)"
```

```
            r"\\\midrule" + "\n")
    (OUT / "centers.tex").write_text(header + "\n".join(rows) + footer)
    pieces = []
    for name, value in [("Afinal", A), ("Afree", Afree),
                        ("bfinal", b[:, None]), ("bfree", bfree[:, None])]:
        pieces.append("\\newcommand{\\" + name + "}{"
                      + tex_matrix(value, 6, name.startswith("b")) + "}")
    (OUT / "intrinsics.tex").write_text("\n".join(pieces))


def main():
    OUT.mkdir(exist_ok=True)
    validate_math()
    data = np.loadtxt(ROOT / "imgpoints.txt")
    assert data.shape == (432, 2) and np.isfinite(data).all()
    uv, p = data.reshape(9, 48, 2), world_points()
    fits = [homography(p, points) for points in uv]
    Hs = np.array([fit[0] for fit in fits])
    errors = np.array([np.linalg.norm(points - project(H, p), axis=1)
                       for H, points in zip(Hs, uv)])
    Afree, bfree, L, sfree = intrinsic(Hs)
    A, b, _, singular = intrinsic(Hs, zero_skew=True)
    Rs, ts, centers = map(np.array, zip(*(extrinsic(H, A) for H in Hs)))
    X = np.c_[p, np.zeros(len(p))]
    pose_errors, depths = [], []
    for R, t, C, points in zip(Rs, ts, centers, uv):
        np.testing.assert_allclose(R.T @ R, np.eye(3), atol=1e-12, rtol=0)
        np.testing.assert_allclose(np.linalg.det(R), 1., atol=1e-12, rtol=0)
        np.testing.assert_allclose(R @ C + t, 0., atol=1e-10, rtol=0)
        camera_points = X @ R.T + t
        depths.extend(camera_points[:, 2])
        pixels = camera_points @ A.T
        pixels = pixels[:, :2] / pixels[:, 2, None]
        pose_errors.append(np.linalg.norm(points - pixels, axis=1))
    assert min(depths) > 0.
    q = inverse_kinematics()
    null = np.linalg.svd(jacobian(q), full_matrices=True)[2][-1]
    summary = dict(ik_degrees=np.degrees(q).tolist(),
                   ik_position=fk_points(q)[-1].tolist(),
                   ik_null_vector=null.tolist(),
                   homography_error=error_stats(errors),
                   per_view_error=[error_stats(e) for e in errors],
                   intrinsic_zero_skew=A.tolist(),
                   intrinsic_unconstrained=Afree.tolist(),
                   b_zero_skew=b.tolist(), b_unconstrained=bfree.tolist(),
                   camera_centers_mm=centers.tolist(),
                   pose_error=error_stats(np.array(pose_errors)),
                   minimum_depth_mm=float(min(depths)),
                   intrinsic_singular_values=singular.tolist(),
                   unconstrained_singular_values=sfree.tolist(),
                   max_rotation_error=float(max(np.linalg.norm(R.T @ R
                       - np.eye(3)) for R in Rs)),
                   validation="All numerical assertions passed.")
    (OUT / "summary.json").write_text(json.dumps(summary, indent=2) + "\n")
    np.savez(OUT / "calibration.npz", world_points_mm=p, image_points=uv,
             homographies=Hs, A=A, b=b, A_unconstrained=Afree,
             b_unconstrained=bfree, L=L, rotations=Rs, translations_mm=ts,
             camera_centers_mm=centers, homography_errors_px=errors)
    save_figures(p, uv, Hs, errors, Rs, centers)
    write_tables(Hs, Rs, ts, centers, errors, A, b, Afree, bfree)
    print(json.dumps(summary, indent=2))


if __name__ == "__main__":
    main()
```